

Aurora Lighting Control System

High-Level Feature Overview & Architecture

Document Status: Draft

Audience: Developers, Contributors, Technical Reviewers, Future Maintainers

Purpose: Provide a high-level overview of Aurora's goals, architecture, and major subsystems.

1. Introduction

Aurora is a professional-grade lighting control platform for macOS designed for live performance environments. The primary goal is to provide an intuitive, low-latency, highly reliable lighting controller capable of replacing existing commercial show control software while remaining modern, extensible, and developer friendly.

Aurora is designed around three fundamental principles:

- **Performance**
- **Reliability**
- **Usability**

The application should feel instantaneous, even during complex shows with thousands of fixtures, hundreds of cues, and continuous live MIDI input.

2. Design Goals

Aurora is intended to be:

- Fast enough for live production
- Stable enough for mission-critical performances
- Flexible enough for future protocols
- Easy enough for volunteers to operate
- Powerful enough for professional programmers

The software should prioritize deterministic behavior over excessive automation. Every action should be predictable and reversible.

3. Major Features

Show Management

Aurora manages complete lighting shows including:

- Fixtures
- Universes
- Groups
- Presets
- Palettes
- Cue Lists
- Songs
- Shows
- Media assets
- User preferences

Shows are stored in a portable project format that can be version controlled and backed up.

Patch Management

Comprehensive fixture patching including:

- DMX Universes
 - Fixture personalities
 - Address management
 - Duplicate detection
 - Visual patch editor
 - Fixture cloning
 - Batch editing
 - Search and filtering
-

Fixture Library

Aurora maintains a fixture definition library supporting:

- Manufacturers
- Models
- DMX channel layouts
- Fine channels
- Color wheels
- Gobos
- Pan/Tilt capabilities

- Beam parameters

Future support should allow importing existing fixture definitions from common industry formats.

Cue Engine

The cue engine provides:

- Cue lists
- Fade timing
- Delay timing
- Crossfades
- Tracking
- Non-tracking modes
- Cue stacks
- Looping
- Follow cues
- Manual cues

Designed to execute with deterministic timing and minimal latency.

Live Playback

Supports live operation through:

- MIDI notes
 - MIDI CC
 - Keyboard shortcuts
 - OSC (future)
 - Touch interface
 - On-screen buttons
 - External control surfaces
-

Song Mode

Songs organize cues into performance sequences.

Features include:

- Song metadata
- Cue order
- Automatic progression
- Manual progression

- Notes
 - Timing information
 - Performer annotations
-

Workspace Layout

Aurora uses dockable panels similar to professional creative applications.

Potential panels include:

- Fixture Browser
- Patch View
- Cue List
- Song View
- Universe Monitor
- Output Monitor
- MIDI Monitor
- RTP-MIDI Connections
- Inspector
- Timeline
- Color Picker
- Programmer
- Console Output

Users can save custom workspace layouts.

Programmer

Live editing environment supporting:

- Temporary values
 - Highlight
 - Blind mode
 - Fan
 - Align
 - Home
 - Locate
 - Selection tools
 - Attribute editing
-

Color Management

Integrated color tools including:

- RGB
 - RGBW
 - CMY
 - HSV
 - Color temperature
 - Virtual color wheels
 - Palette generation
-

Effect Engine

Supports dynamic effects such as:

- Chase
- Pulse
- Wave
- Rainbow
- Circle
- Random
- Sparkle
- Position effects
- Intensity effects

Effects should be fully parameterized and reusable.

Networking

Native networking includes:

- Built-in RTP-MIDI
- Art-Net (future)
- sACN/E1.31 (future)
- OSC (future)
- Remote control APIs

Networking should avoid unnecessary external dependencies wherever practical.

MIDI System

Integrated MIDI subsystem supporting:

- CoreMIDI
 - Built-in RTP-MIDI
 - MIDI Notes
 - CC
 - Program Change
 - MIDI Learn
 - Device mapping
 - Multiple devices simultaneously
-

Monitoring & Diagnostics

Operational tools include:

- DMX Output Monitor
 - Universe Viewer
 - Packet Monitor
 - MIDI Monitor
 - Network diagnostics
 - Logging
 - Performance metrics
-

Undo / Redo

Nearly every editing action should support:

- Unlimited undo
 - Unlimited redo
 - Transaction grouping
 - Persistent history during editing sessions
-

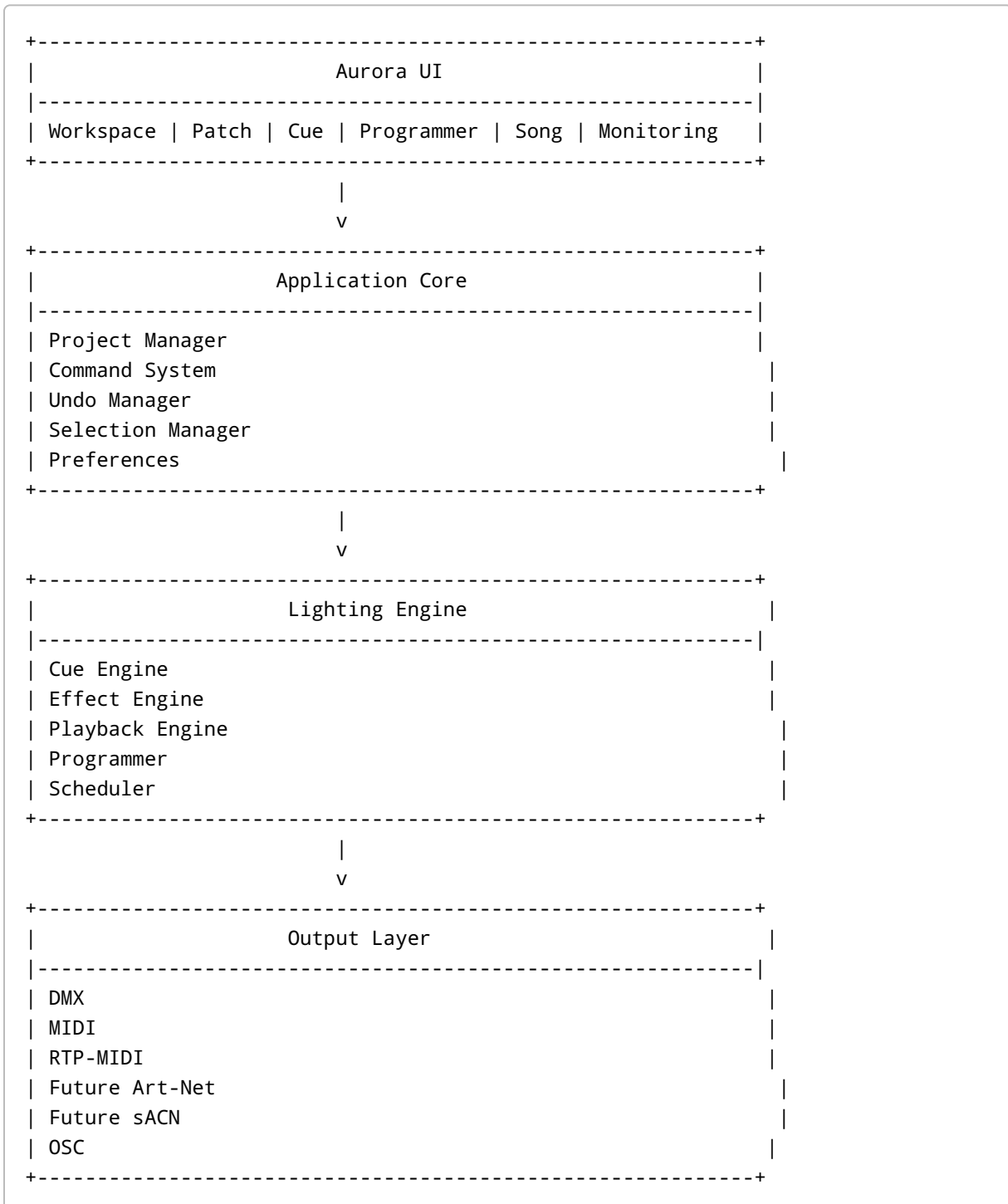
Plugin Architecture

Future versions may support plugins for:

- New protocols
- Import/export formats
- Effect generators
- Fixture libraries

- Scripting
- Hardware integrations

4. High-Level System Architecture



5. Core Architectural Principles

Modular

Every major subsystem should operate independently with clearly defined interfaces.

Benefits include:

- Easier testing
- Easier maintenance
- Faster development
- Plugin compatibility

Event Driven

Aurora should be heavily event based.

Examples:

- MIDI event received
- Cue started
- Cue completed
- Fixture updated
- Universe changed
- Project modified

This minimizes coupling between systems.

Command Pattern

User actions should execute as commands.

Examples:

- Add Fixture
- Delete Cue
- Patch Fixture
- Rename Song
- Move Cue

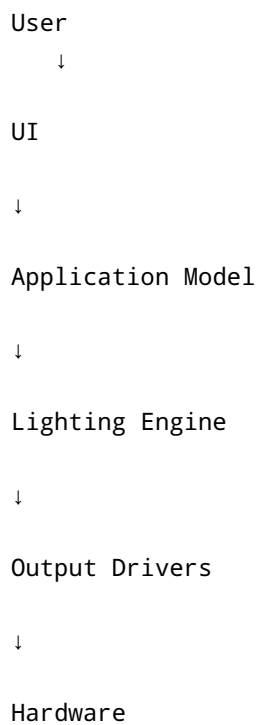
Benefits:

- Undo
- Redo
- History
- Macro recording (future)

Data Model Separation

UI should never directly manipulate hardware.

Instead:



This separation improves reliability and enables offline editing.

6. Performance Goals

Aurora should target:

- Instant UI response
- Minimal MIDI latency

- Stable timing
- Efficient memory usage
- Large project scalability
- Smooth rendering
- Non-blocking background tasks

Performance is considered a first-class feature rather than a post-development optimization.

7. Future Expansion

The architecture is intentionally designed to support future capabilities such as:

- Multi-user collaboration
 - Remote programming
 - Tablet companion applications
 - Timeline-based programming
 - Timecode synchronization
 - Audio analysis
 - AI-assisted cue generation
 - Integrated visualizer
 - Pixel mapping
 - Video playback synchronization
 - Plugin marketplace
-

8. Guiding Philosophy

Aurora should feel like a modern professional creative application rather than a traditional hardware lighting console.

The emphasis is on clarity, responsiveness, and maintainability while providing the depth expected by advanced users. Every subsystem should be extensible, every interaction should be predictable, and every architectural decision should favor long-term reliability over short-term convenience.

This document serves as the foundation for the detailed design documents that follow, each describing a specific subsystem, data model, API, or implementation strategy.